Type-wrapper classes

- Type-wrapper classes allow to use primitive data types where only objects can be used. Example:

// DEMO ONLY

```
public class Main {
      void main() {
            Object [] ao = new Object[3];
            ao[0] = new Fraction(1, 2);     //ok
            ao[1] = new Point(2, 3);        // ok
            ao[2] = 1;                      // ????

      }
}
```

1

Type-wrapper classes

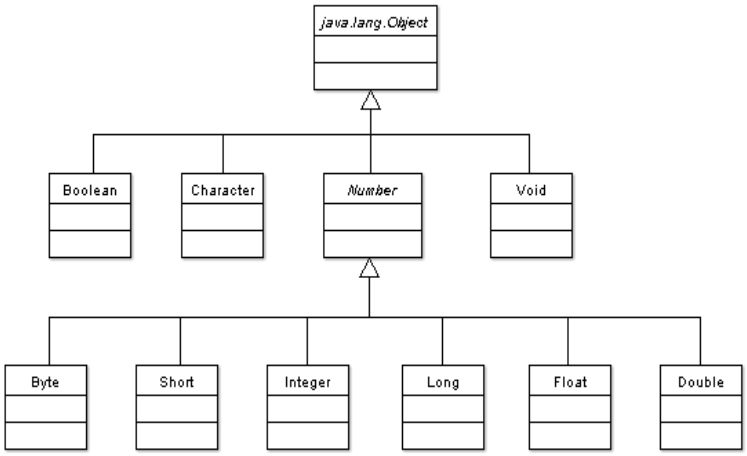
- Type-wrapper classes allow to use primitive data types where only objects can be used. Example:

```
public class Main {
   void main() {
            Queue<Integer> q = new LinkedList<Integer>();
            q.add(12);
            q.add(25);
            while(!q.isEmpty())
                  System.out.println(q.remove());
   }
}
```

12 -> Integer.valueOf(12)

2

The hierarchy of type-wrapper classes



3

Type-wrapper classes

- Typically, a type-wrapper class implements library functions and store constant values associated with the type it represents.

  Example: The class type-wrapper class Character implements methods such as

  static boolean isDigit(char c)

  and store constants such as

  MAX_VALUE

  which containts '\uFFFF', the maximum value of primitive data type char.

4

Type-wrapper classes

```
class Integer extends Number {
  public static int parseInt (String str);
  //…
}
```

```
String s = new String("10");
int n = Integer.parseInt(s);
```

5

The Java abstract class Object, main methods

```
protected Object clone()
        throws CloneNotSupportedException

public boolean equals(Object obj)

protected void finalize()  throws Throwable

public final Class<?> getClass()

public String toString()
```

6

The Java abstract class Object, main methods

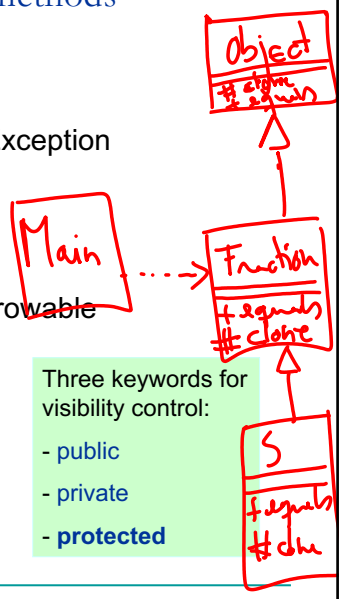
protected Object clone()
throws CloneNotSupportedException

public boolean equals(Object obj)

protected void finalize() throws Throwable

public final Class<?> getClass()

public String toString()



Three keywords for visibility control:

- public
- private
- **protected**

7



Rules for method overriding

Credit: ESA/CNES/Arianespace – Photo Optique Video du CSG – P. Baudon

8

The Java abstract class Object, main methods

protected Object clone()
throws CloneNotSupportedException

public boolean equals(Object obj)

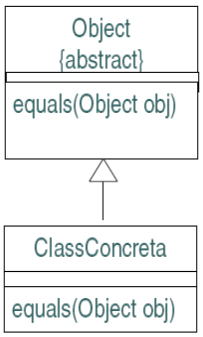
protected void finalize() throws Throwable

public final Class<?> getClass()

public String toString()

9

Overriding unwanted inherited behaviour



Checks whether **this** and **obj** are the same object,

i.e., returns (this==obj)

10

Overriding unwanted inherited behaviour, another idiom

```
/** @version 1.2 */
public class Fraction {
    private int num, den;
    //… As before …

    @Override
    public boolean equals(Object other) {
      if (other == this) return true;
      if (other == null) return false;
      if (getClass() != other.getClass()) return false;
      Fraction that = (Fraction) other;
      return (this.num == that.num && this.den == that.den);
  }}
```

11

Rules for method overriding

- It is possible to redefine a method in a subclass in a way that is inconsistent with the superclass method

  Use class invariants, pre and post conditions to prevent this:

  - Subclass method's pre-conditions cannot be more demanding, but can be softned
  - Class invariant, as well as post-conditions of a class methods should aply also to its subclasses. Subclasses might ensure stronger post-conditions but they cannot relax them.

12

Rules for method overriding

- Specially:

  The visibility of a overrided method can be different from the superclass method; however visibility can only increase

  Example: a protected method can be override in a subclass class as protected or public; never as private.

13

Visibility of equals in Object

```
protected Object clone()
        throws CloneNotSupportedException

public boolean equals(Object obj)

protected void finalize()  throws Throwable

public final Class<?> getClass()

public String toString()
```

14

Overriding unwanted inherited behaviour, another idiom

```
/** @version 1.2 */
public class Fraction {
    private int num, den;
    //… As before …

    @Override
    public boolean equals(Object other) {
        if (other == this) return true;
        if (other == null) return false;
        if (getClass() != other.getClass()) return false;
        Fraction that = (Fraction) other;
        return (this.num == that.num && this.den == that.den);
  }}
```

15

Rules for method overriding

A subclass shoud not ask more nor offer less than its superclass

16

Rules for method overriding

- Subclass method's pre-conditions cannot be more restrictive, but can be softned
- Class invariant, as well as post-conditions of a class methods should aply also to its subclasses.

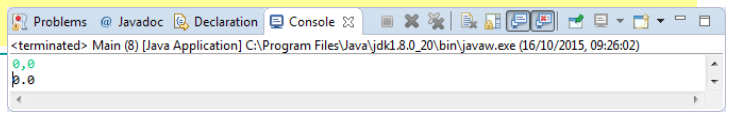
  Subclasses might ensure stronger post-conditions but they cannot relax them.

17

Rules for method overriding – Example

```
public class A {
        /**   @pre x >= 0
              @return y; s.t. y*y == x
              @post y >=0 */
        public double f(double x) {  return Math.sqrt(x); } }
```

```
public class Main {
        void main() {
                Scanner s = new Scanner (System.in);
                double d = s.nextDouble();
                A x = new A();
                if (d >=0) // Ensures A.f pre-condition
                    System.out.println(x.f(d));
        } }
```



18

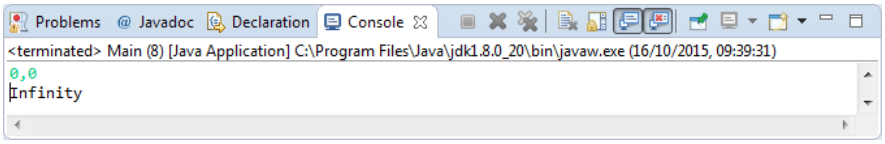
Rules for method overriding – Example

```
public class A {                                  // As before
        /**  @pre x >= 0
             @return y; s.t. y*y == x
             @post y >=0 */
        public double f(double x) {  return Math.sqrt(x); } }

class B extends A {
        /** @pre x > 0  */                  // WRONG method overriding
        public  double f(double x) {    // pre-conditon of a overrided
                double y = 1/x;         // cannot be more restrictive
                return Math.sqrt(y);    // than the original method
        } }
```

19

Rules for method overriding – Example




```
public class Main {
        void main() {
                Scanner s = new Scanner (System.in);
                double d = s.nextDouble();
                A x = new B();
                if (d >=0) // Ensures A.f pre-condition
                    System.out.println(x.f(d));
        } }
```

20

Rules for method overriding

- Subclass method's pre-conditions cannot be more restrictive, but can be softned
- Class invariant, as well as post-conditions of a class methods should aply also to its subclasses.

Subclasses might ensure stronger post-conditions but they cannot relax them.

21

Rules for method overriding – Example 2

```
public class A {
        /**   @pre x > 0
              @return y; s.t. 1/y/y == x
              @post y > 0  */
        public double f(double x) {  return Math.sqrt(1.0/x); }  }
```

```
public class Main {
        void main() {
                Scanner s = new Scanner (System.in);
                double d = s.nextDouble();
                A x = new A();
                double positive = d*d +1.0;
                double ans = x.f(positive);  // assert ans > 0
                System.out.println(1/ans);
        } }
```

22

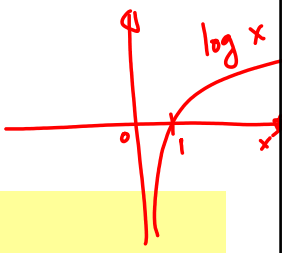
Rules for method overriding – Example 2

```
public class A {                                  //As before
        /**   @pre x > 0
              @return y; s.t. 1/y/y == x
              @post y > 0  */
        public double f(double x) {  return Math.sqrt(1.0/x); } }

class B extends A {
        /** @pre x > 0
            @post y in   [Double.NEGATIVE_INFINITY;
                          Double.POSITIVE_INFINITY];
        public  double f(double x) {     // WRONG method overriding
                return Math.log(x);      // post-conditon of a overrided
        }                                // cannot be more general
 }                                       // than the original method
```

23

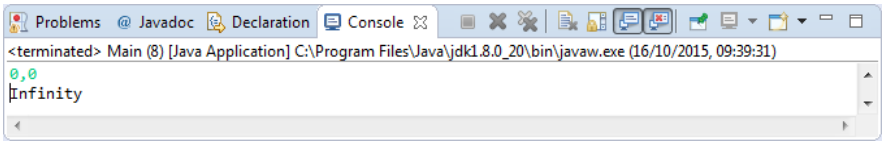
Rules for method overriding – Example 2




```
public class Main {
        void main() {
                Scanner s = new Scanner (System.in);
                double d = s.nextDouble();
                A x = new A();
                double positive = d*d +1.0;
                double ans = x.f(positive);  // assert ans > 0
                System.out.println(1/ans);
        } }
```

24

Rules for method overriding – Example 2




```
public class Main {
        void main() {
                Scanner s = new Scanner (System.in);
                double d = s.nextDouble();
                A x = new B();
                double positive = d*d +1.0;
                double ans = x.f(positive);  // assert ans > 0
                System.out.println(1/ans);  // No longer valid
        } }
```

25

"Reuse without a contract is sheer folly"

The contract, i.e., invariant, pre and post conditions of a class, should be part of the standard documentation

Jean-Marc Jézéquel, IRISA
Bertrand Meyer, ISE

26

"Reuse without a contract is sheer folly"

"For reuse to be effective, design by contract is a requirement.

Without a precise specification attached to each reusable component - precondition, post condition, invariant - no one can trust a supposedly reusable component"

Jean-Marc Jézéquel, IRISA
Bertrand Meyer, ISE

27

Example: Ariane 5

- https://pvs-studio.com/en/blog/posts/cpp/0426/
- https://www.reddit.com/r/ProgrammerHumor/comments/ar0v0c/370_million_for_an_integer_overflow_the_june_4/



28

Example: Ariane 5

- **Conclusion**



- The defect on the Ariane 5 was the result of several factors.
- There **were many stages during development and testing when the defect could have been detected**.
- (...)

29

Check point: Is this override possible?

```
/** @version 1.2 */
public class Fraction {

    @Override
    private Object clone() {
        System.out.print("humm…");
        return super.clone();
    }
}
```

Object {abstract}
clone(): Object

```
public static void main(String []) {
    Fraction f = new Fraction(1, 2);

    // But can we use it like this?

    Fraction g = (Fraction) f.clone(); }
```

30

Where are we?

- Class hierarchies
- Abstract classes
- The Java Object class
- The type wrapper classes of Java
- Rules for method overriding

31